

Review of Last Class

1. Neural fields and coordinate-based scene representations
2. Classical rendering fundamentals: radiometry, BRDF, and the rendering equation
3. Ray tracing and rendering acceleration techniques
4. Two seminal works: Scene Representation Networks (SRN) and Neural Volumes and volumetric neural rendering

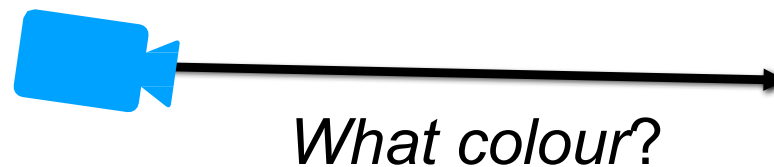
Neural Radiance Fields (NeRF)

Mildenhall, Srinivasan, Tancik, Barron, Ramamoorthi, Ng
ECCV 2020

Neural Volumetric Rendering

Neural Volumetric Rendering

querying the radiance value
along rays through 3D space



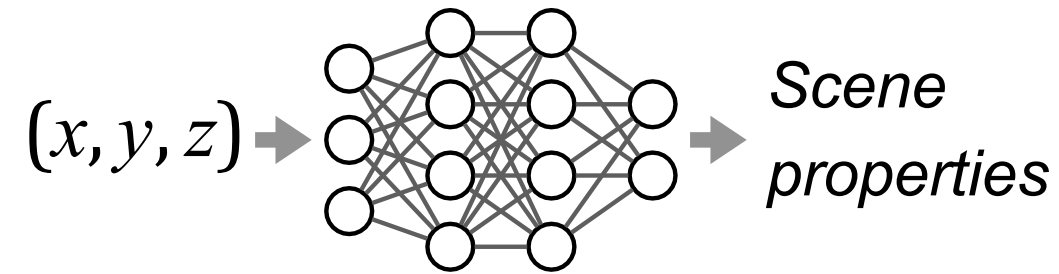
Neural Volumetric Rendering

continuous, differentiable
rendering model without
concrete ray/surface intersections



Neural Volumetric Rendering

using a neural network as a
scene representation, rather
than a voxel grid of data





Inputs: sparse, unstructured
photographs of a scene

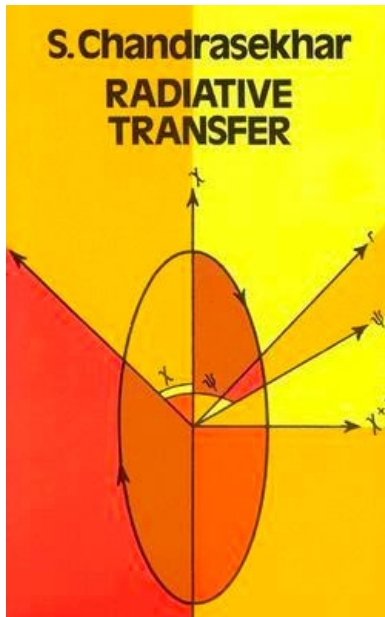


Outputs: representation allowing us to
render *new* views of that scene

Overview of NeRF

- ▶ Volumetric rendering math
- ▶ Neural networks as representations for spatial data
- ▶ Neural Radiance Fields (NeRF)

Traditional volumetric rendering



- ▶ Theory of volume rendering co-opted from physics in the 1980s: absorption, emission, out-scattering/in-scattering
- ▶ Adapted for visualising medical data and linked with alpha compositing
- ▶ Modern path tracers use sophisticated Monte Carlo methods to render volumetric effects

Chandrasekhar 1950, *Radiative Transfer*

Kajia 1984, *Ray Tracing Volume Densities*

Levoy 1988, *Display of Surfaces from Volume Data*

Max 1995, *Optical Models for Direct Volume Rendering*

Porter and Duff 1984, *Compositing Digital Images*

Novak et al 2018, *Monte Carlo methods for physically based volume rendering*

Traditional volumetric rendering



Medical data visualisation
[Levoy]

- ▶ Theory of volume rendering co-opted from physics in the 1980s: absorption, emission, out-scattering/in-scattering
- ▶ Adapted for visualising medical data and linked with alpha compositing
- ▶ Modern path tracers use sophisticated Monte Carlo methods to render volumetric effects

Alpha compositing [Porter and Duff]

Chandrasekhar 1950, *Radiative Transfer*

Kajia 1984, *Ray Tracing Volume Densities*

Levoy 1988, *Display of Surfaces from Volume Data*

Max 1995, *Optical Models for Direct Volume Rendering*

Porter and Duff 1984, *Compositing Digital Images*

Novak et al 2018, *Monte Carlo methods for physically based volume rendering*

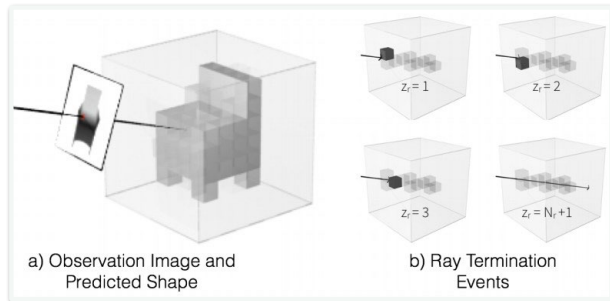
Traditional volumetric rendering



Physically-based Monte Carlo rendering [Novak et al]

- ▶ Theory of volume rendering co-opted from physics in the 1980s: absorption, emission, out-scattering/in-scattering
- ▶ Adapted for visualising medical data and linked with alpha compositing
- ▶ Modern path tracers use sophisticated Monte Carlo methods to render volumetric effects

Volumetric rendering and machine learning



"Probabilistic" voxel grid rendering [Tulsiani et al]

- ▶ Various volume-rendering methods devised for 3D shape reconstruction methods
- ▶ Scaled up to higher resolution volumes to achieve excellent view synthesis results

Tulsiani et al 2017, *Multi-view Supervision for Single-view Reconstruction via Differentiable Ray Consistency*

Henzler et al 2019, *Escaping Plato's Cave: 3D Shape From Adversarial Rendering*

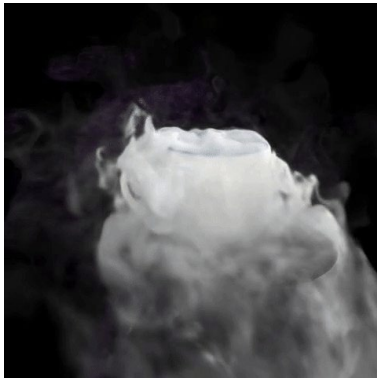
Zhou et al 2018, *Stereo Magnification: Learning View Synthesis using Multiplane Images*

Lombardi et al 2019, *Neural Volumes: Learning Dynamic Renderable Volumes from Images*

Volumetric rendering and machine learning



Slices from a volumetric scene representation [Zhou et al]



View synthesis from a dynamic voxel grid [Lombardi et al]

- ▶ Various volume-rendering-esque methods devised for 3D shape reconstruction methods
- ▶ Scaled up to higher resolution voxel grids, ML methods can achieve excellent view synthesis results

Tulsiani et al 2017, *Multi-view Supervision for Single-view Reconstruction via Differentiable Ray Consistency*

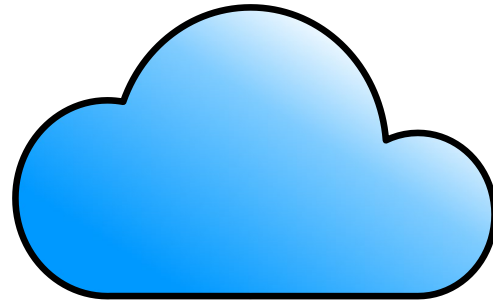
Henzler et al 2019, *Escaping Plato's Cave: 3D Shape From Adversarial Rendering*

Zhou et al 2018, *Stereo Magnification: Learning View Synthesis using Multiplane Images*

Lombardi et al 2019, *Neural Volumes: Learning Dynamic Renderable Volumes from Images*

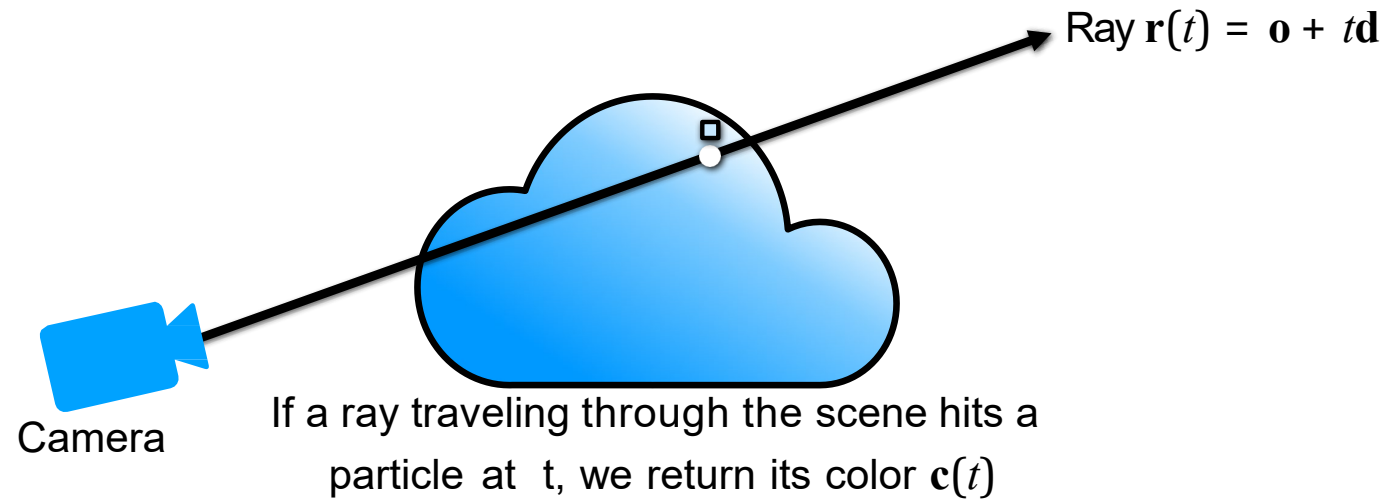
Volumetric formulation for NeRF

Volumetric formulation for NeRF

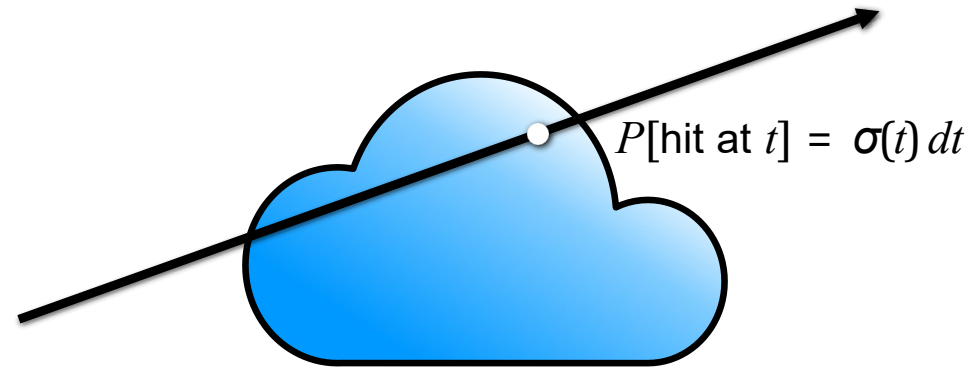


Scene is a cloud of tiny colored particles

Volumetric formulation for NeRF

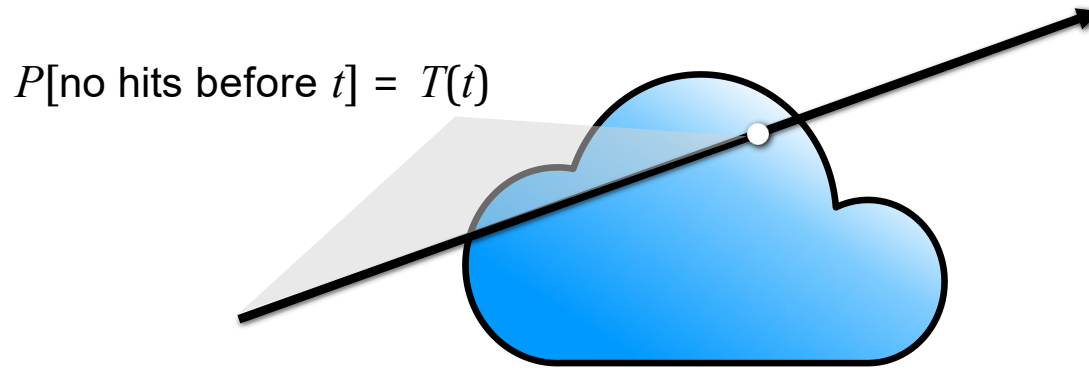


Volumetric formulation for NeRF



This notion is *probabilistic*: chance that ray stops in a small interval around t is $\sigma(t) dt$.
 $\sigma(t)$ is known as the “volume density”

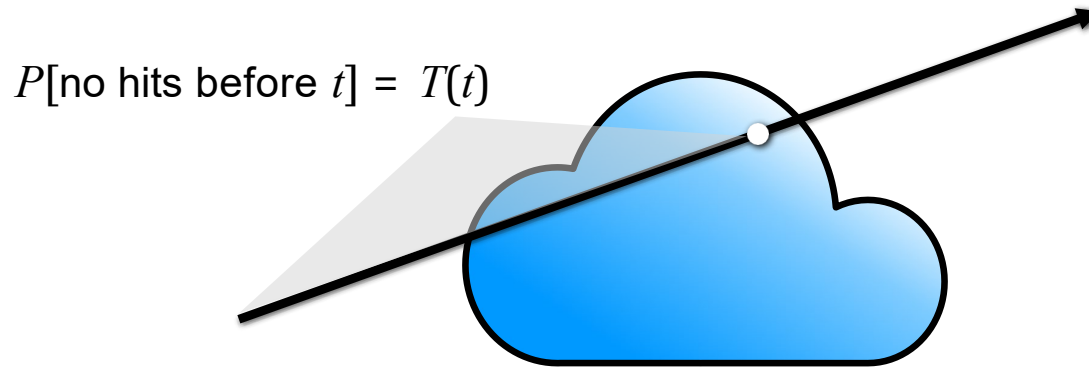
Volumetric formulation for NeRF



To determine if t is the *first* hit, need to know $T(t)$:
probability that the ray didn't hit any particles earlier.

$T(t)$ is called “transmittance”

Volumetric formulation for NeRF

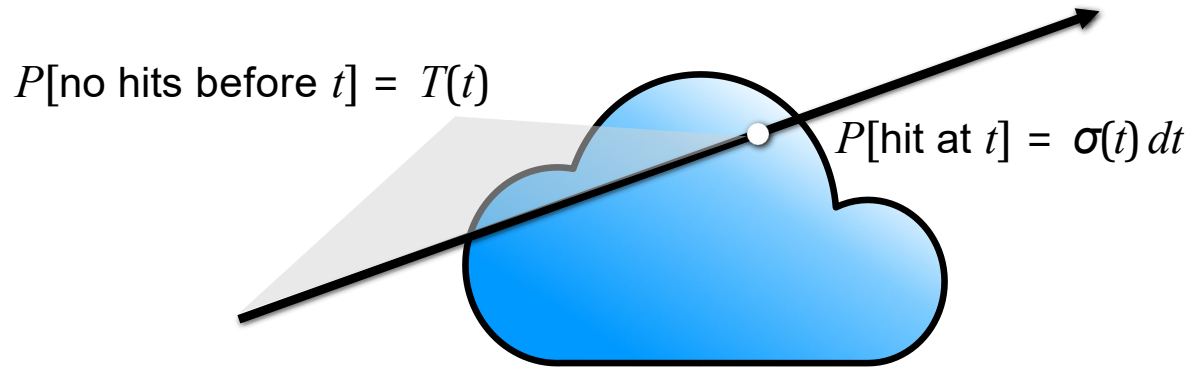


To determine if t is the *first* hit, need to know $T(t)$:
probability that the ray didn't hit any particles earlier.

$T(t)$ is called “transmittance”

We assume σ is known and want to use it to calculate $T(t)$

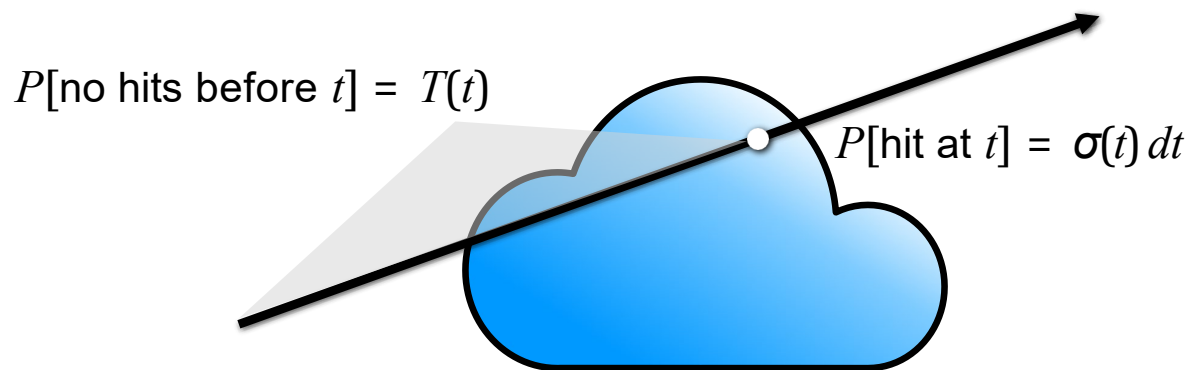
Volumetric formulation for NeRF



σ and T are related by the probability fact that

$$P[\text{no hits before } t + dt] = P[\text{no hits before } t] \times P[\text{no hit at } t]$$

Volumetric formulation for NeRF



These are related by the probability fact that

$$T(t + dt) = T(t) \times (1 - \sigma(t)dt)$$

Volumetric formulation for NeRF

$$T(t + dt) = T(t)(1 - \sigma(t)dt)$$

Volumetric formulation for NeRF

$$T(t + dt) = T(t)(1 - \sigma(t)dt)$$

Split up differential $\Rightarrow$ $T(t) + T'(t)dt = T(t) - T(t)\sigma(t)dt$

Volumetric formulation for NeRF

$$T(t + dt) = T(t)(1 - \sigma(t)dt)$$

Split up differential $\Rightarrow T(t) + T'(t)dt = T(t) - T(t)\sigma(t)dt$

Rearrange $\Rightarrow \frac{T'(t)}{T(t)} dt = -\sigma(t)dt$

Volumetric formulation for NeRF

$$T(t + dt) = T(t)(1 - \sigma(t)dt)$$

Split up differential $\Rightarrow T(t) + T'(t)dt = T(t) - T(t)\sigma(t)dt$

Rearrange $\Rightarrow \frac{T'(t)}{T(t)} dt = -\sigma(t)dt$

Integrate $\Rightarrow \log T(t) = -\int_{t_0}^t \sigma(s)ds$

Volumetric formulation for NeRF

Thus, the probability that a ray first hits a particle at t is

$$T(t)\sigma(t) dt = \exp\left(-\int_{t_0}^t \sigma(t)\right) \sigma(t) dt$$

Volumetric formulation for NeRF

Thus, the probability that a ray first hits a particle at t is

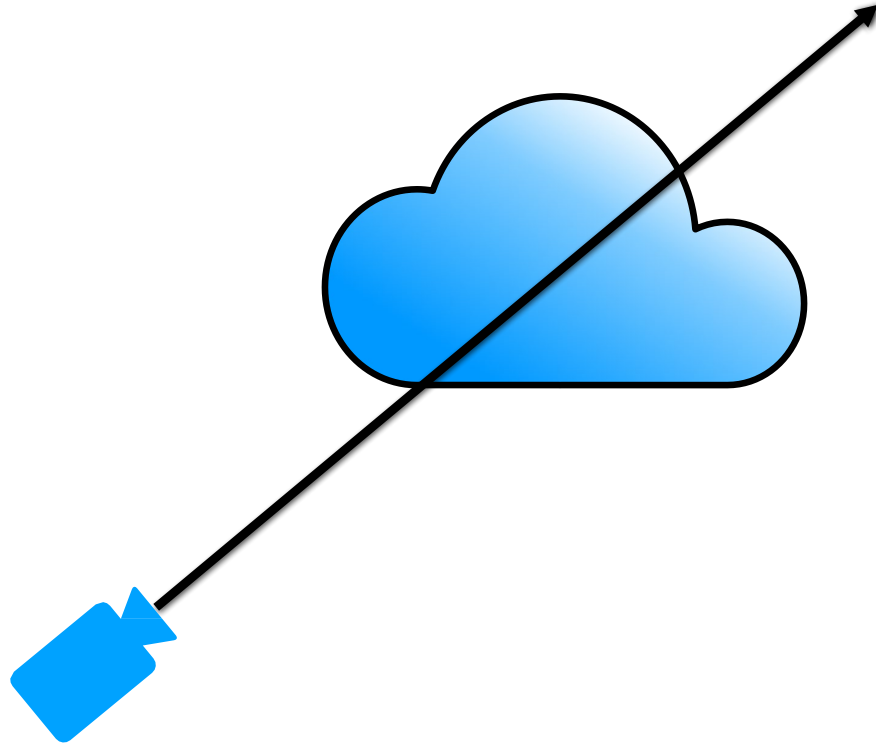
$$T(t)\sigma(t) dt = \exp\left(-\int_{t_0}^t \sigma(t)\right) \sigma(t) dt$$

And expected color returned by the ray will be

$$\int_{t_{near}}^{t_{far}} T(t)\sigma(t)\mathbf{c}(t) dt$$

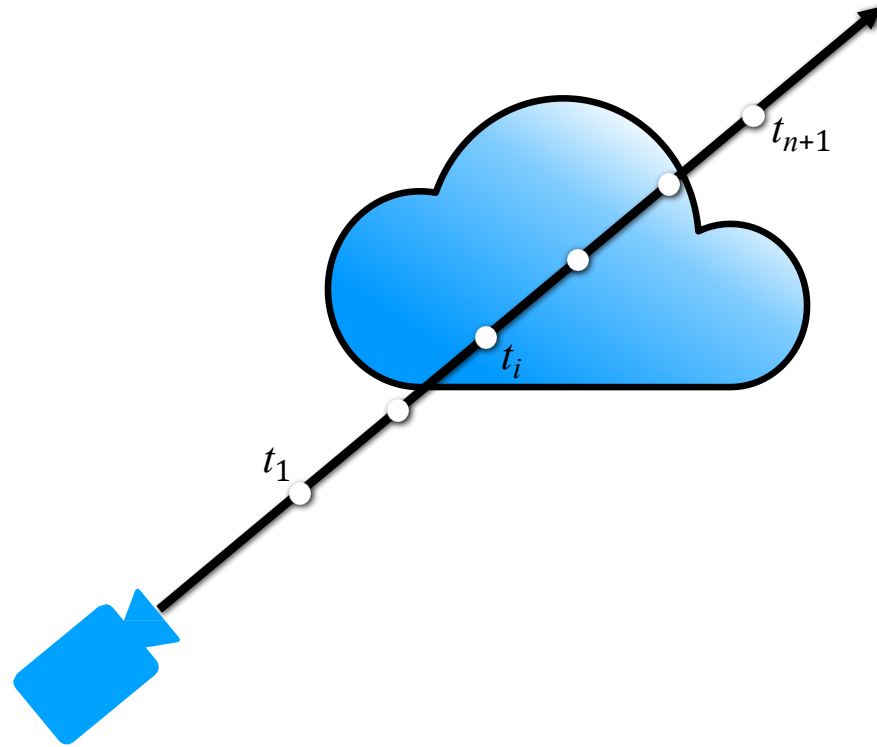
Note the nested integral!

Approximating the nested integral



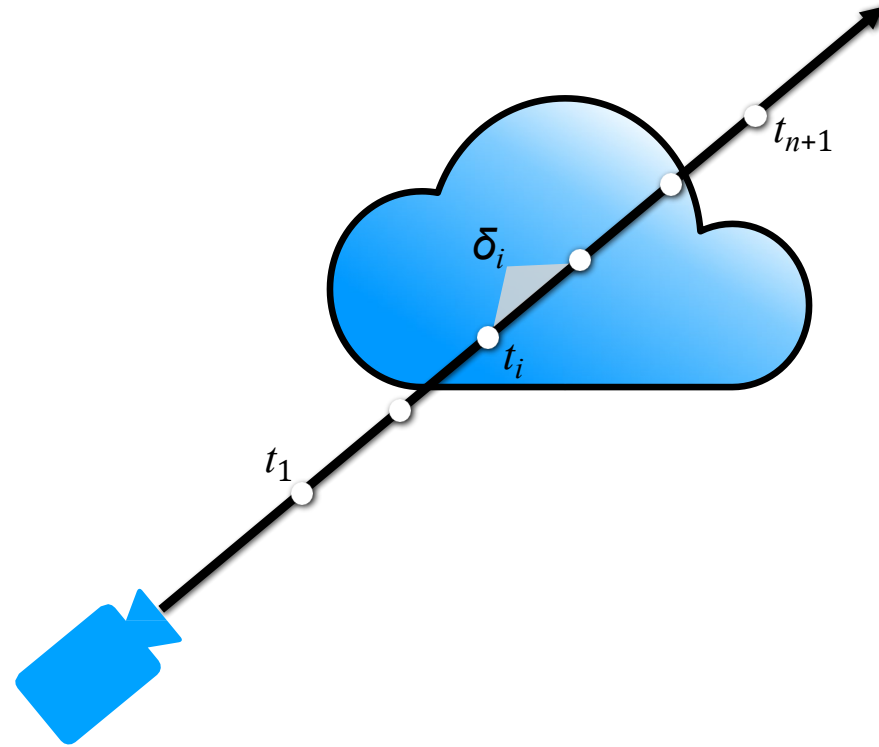
We use quadrature to approximate the nested integral,

Approximating the nested integral



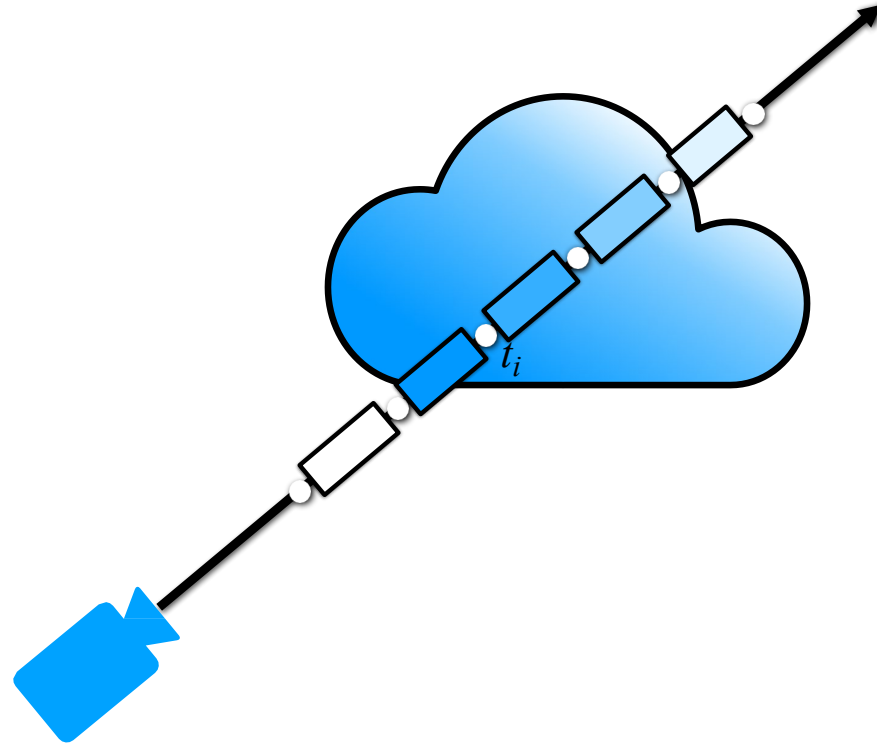
We use quadrature to approximate the nested integral, splitting the ray up into n segments with endpoints $\{t_1, t_2, \dots, t_{n+1}\}$

Approximating the nested integral



We use quadrature to approximate the nested integral,
splitting the ray up into n segments with endpoints $\{t_1, t_2, \dots, t_{n+1}\}$
with lengths $\delta_i = t_{i+1} - t_i$

Approximating the nested integral



We assume volume density and color are roughly constant within each interval

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt$$

This allows us to break the outer integral

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt \approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} T(t)\sigma_i\mathbf{c}_i dt$$

This allows us to break the outer integral into a sum of analytically tractable integrals

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt \approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} \boxed{T(t)\sigma_i\mathbf{c}_i} dt$$

Catch: piecewise constant density and color
do not imply constant transmittance!

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt \approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} \boxed{T(t)} \sigma_i \mathbf{c}_i dt$$

Catch: piecewise constant density and color
do not imply constant transmittance!

Important to account for how early part of a
segment blocks later part when σ_i is high

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt \approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} T(t)\sigma_i\mathbf{c}_i dt$$

For $t \in [t_i, t_{i+1}]$, $T(t) = \exp\left(-\int_{t_1}^{t_i} \sigma_i ds\right) \exp\left(-\int_{t_i}^t \sigma_i ds\right)$

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt \approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} T(t)\sigma_i\mathbf{c}_i dt$$

For $t \in [t_i, t_{i+1}]$, $T(t) = \exp\left(-\int_{t_1}^{t_i} \sigma_i ds\right) \exp\left(-\int_{t_i}^t \sigma_i ds\right)$


$$\exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right) = T_i$$

“How much is blocked by all previous segments?”

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt \approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} T(t)\sigma_i \mathbf{c}_i dt$$

For $t \in [t_i, t_{i+1}]$, $T(t) = \exp\left(-\int_{t_1}^{t_i} \sigma_i ds\right) \exp\left(-\int_{t_i}^t \sigma_i ds\right)$

"How much is blocked partway through the current segment?"


$$\exp(-\sigma_i(t - t_i))$$

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt \approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} T(t)\sigma_i\mathbf{c}_i dt$$

Substitute

$$= \sum_{i=1}^n T_i\sigma_i\mathbf{c}_i \int_{t_i}^{t_{i+1}} \exp(-\sigma_i(t - t_i)) dt$$

Approximating the nested integral

$$\int T(t)\sigma(t)\mathbf{c}(t) dt \approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} T(t)\sigma_i\mathbf{c}_i dt$$

$$= \sum_{i=1}^n T_i\sigma_i\mathbf{c}_i \int_{t_i}^{t_{i+1}} \exp(-\sigma_i(t - t_i)) dt$$

Integrate

$$= \sum_{i=1}^n T_i\sigma_i\mathbf{c}_i \frac{\exp(-\sigma_i(t_{i+1} - t_i)) - 1}{-\sigma_i}$$

Approximating the nested integral

$$\begin{aligned}\int T(t)\sigma(t)\mathbf{c}(t) dt &\approx \sum_{i=1}^n \int_{t_i}^{t_{i+1}} T(t)\sigma_i\mathbf{c}_i dt \\&= \sum_{i=1}^n T_i\sigma_i\mathbf{c}_i \int_{t_i}^{t_{i+1}} \exp(-\sigma_i(t - t_i)) dt \\&= \sum_{i=1}^n T_i\sigma_i\mathbf{c}_i \frac{\exp(-\sigma_i(t_{i+1} - t_i)) - 1}{-\sigma_i} \\&\quad \text{Cancel } \sigma_i \\&= \sum_{i=1}^n T_i\mathbf{c}_i (1 - \exp(-\sigma_i\delta_i))\end{aligned}$$

Connection to alpha compositing

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$$



$$= \sum_{i=1}^n T_i \mathbf{c}_i (1 - \exp(-\sigma_i \delta_i))$$

Connection to alpha compositing

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i) \Rightarrow$$

$$= \sum_{i=1}^n T_i \mathbf{c}_i (1 - \exp(-\sigma_i \delta_i))$$


$$\text{color} = \sum_{i=1}^n T_i \alpha_i \mathbf{c}_i = \sum_{i=1}^n T_i \mathbf{c}_i (1 - \exp(-\sigma_i \delta_i))$$

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j) = \exp \left(- \sum_{j=1}^{i-1} \sigma_j \delta_j \right)$$

Summary: volume rendering integral estimate

Rendering model for ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$:

$$\mathbf{c} \approx \sum_{i=1}^n T_i \alpha_i \mathbf{c}_i$$

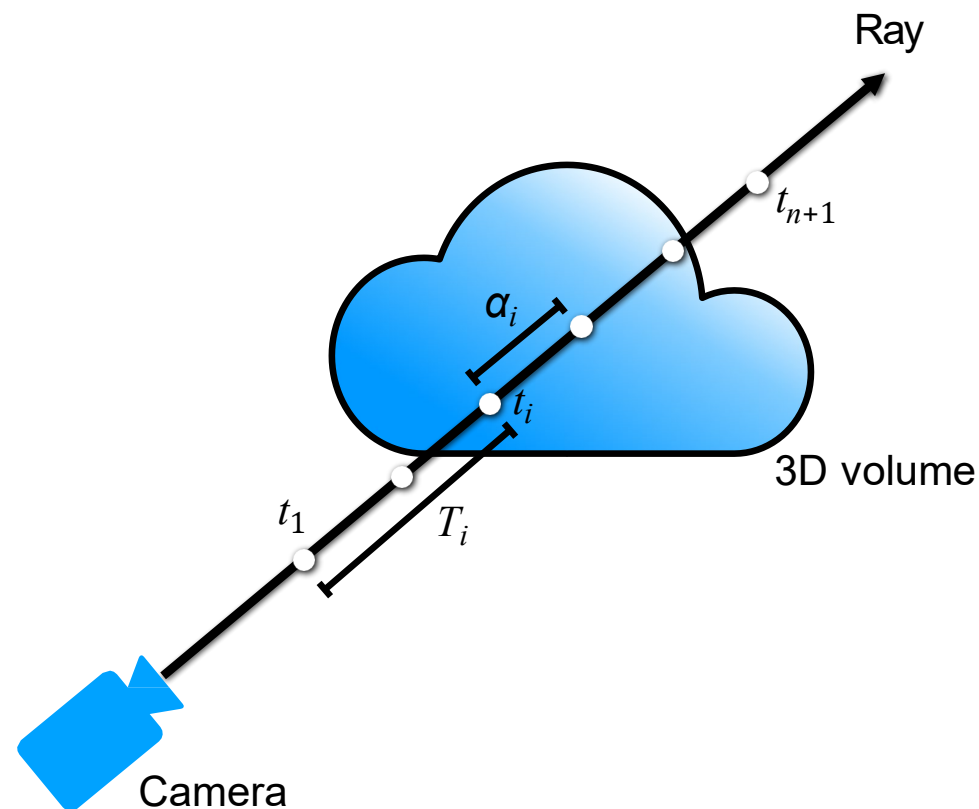
weights colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

How much light is contributed by ray segment i :

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$$



Summary: volume rendering integral estimate

Rendering model for ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$:

$$\mathbf{c} \approx \sum_{i=1}^n T_i \alpha_i \mathbf{c}_i$$

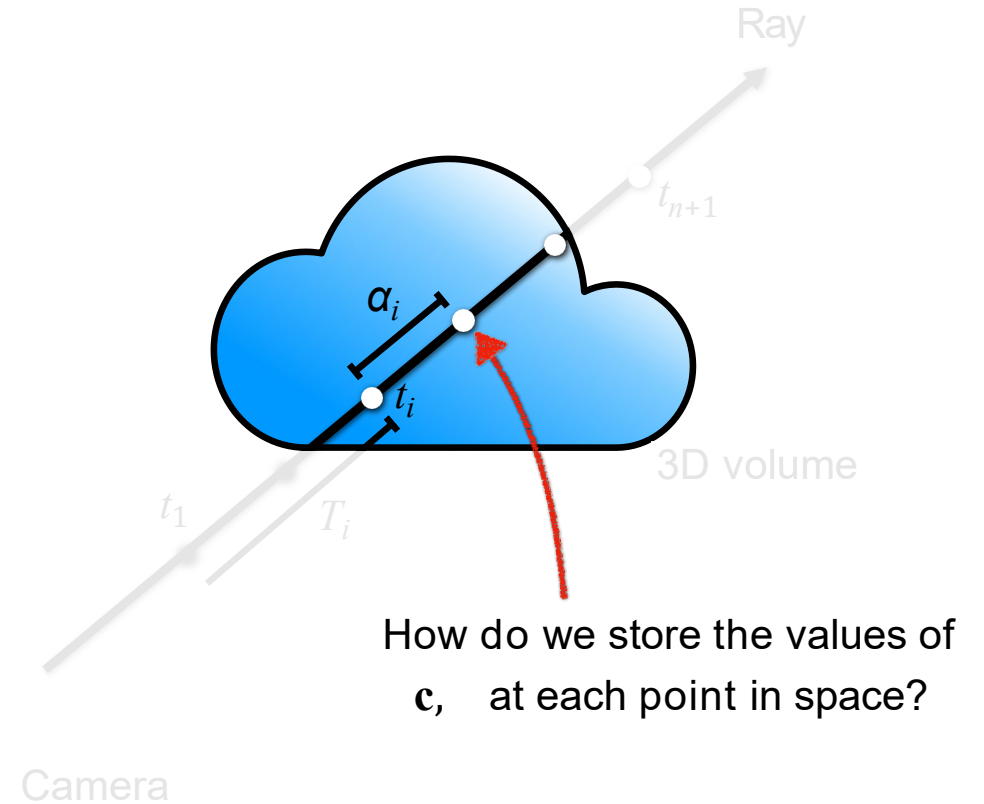
weights colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

How much light is contributed by ray segment i :

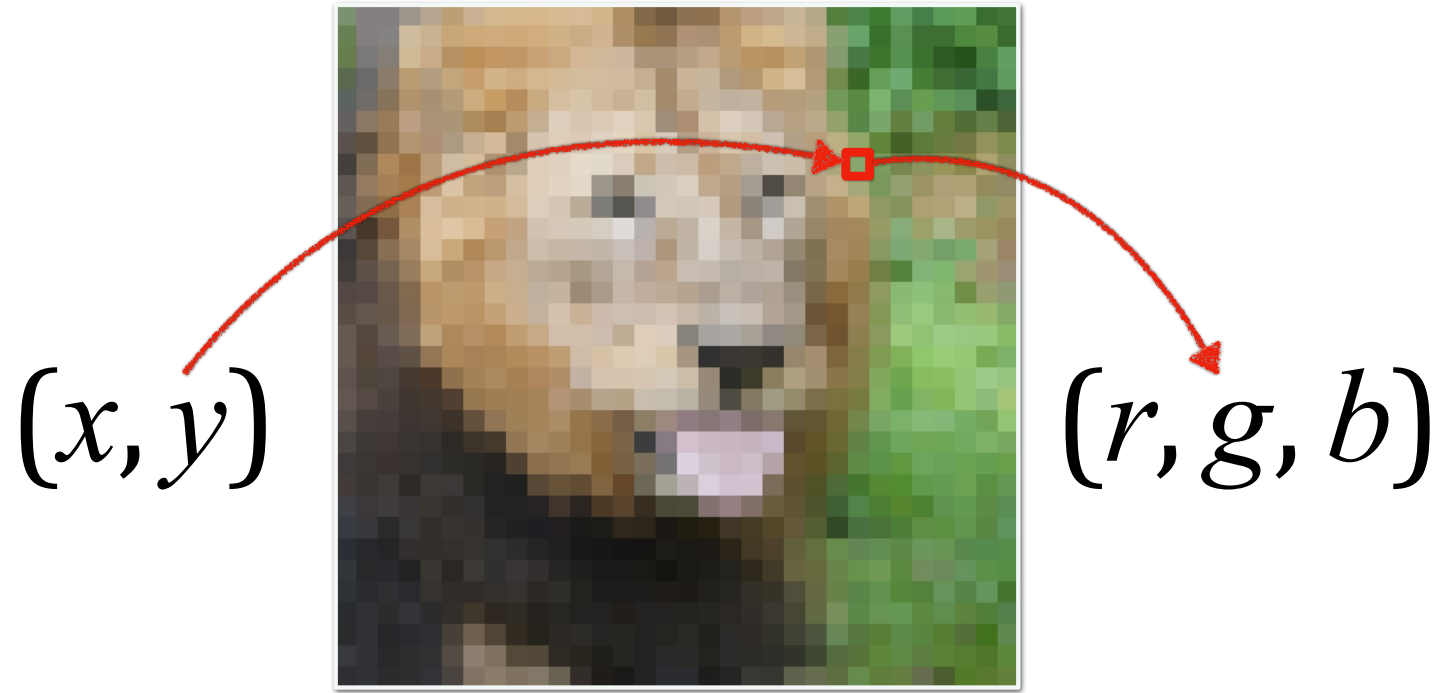
$$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$$



Overview

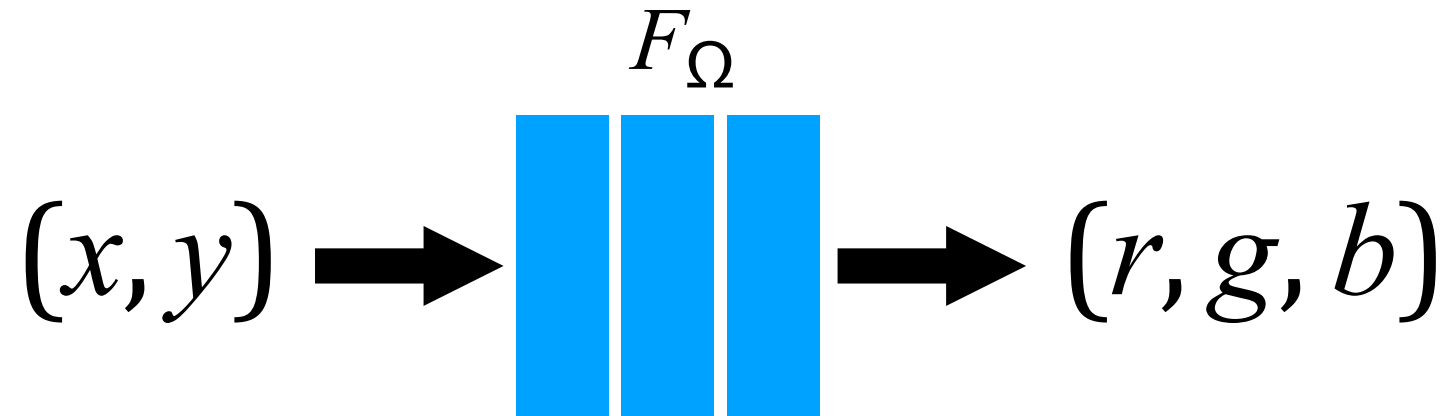
- ▶ Volumetric rendering math
- ▶ **Neural networks as representations for spatial data**
- ▶ Neural Radiance Fields (NeRF)

Toy problem: storing 2D image data



Usually we store an image as a
2D grid of RGB color values

Toy problem: storing 2D image data



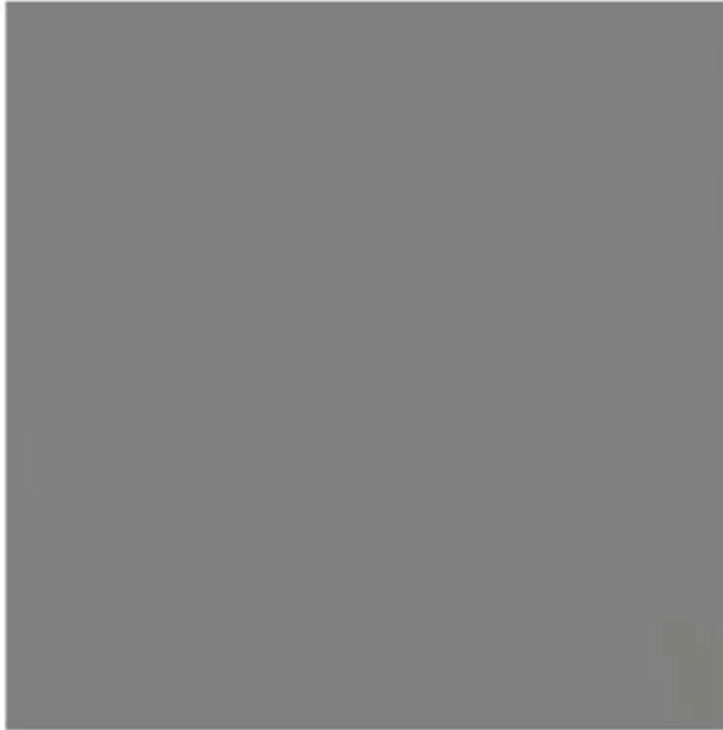
What if we train a simple fully-connected network (MLP) to do this instead?

Naive approach fails!

Ground truth image



Standard fully-connected net



Problem:

“Standard” coordinate-based MLPs cannot represent high-frequency functions

Solution:

Pass input coordinates through a
high frequency mapping first

Input coordinate mapping

- ▶ Simple formula: apply a tall skinny matrix $\mathbf{B}$ to input coordinate vector $\mathbf{x}$, then pass through *sin* and *cos*:

$$\gamma(\mathbf{x}) = (\sin(2\pi\mathbf{B}\mathbf{x}), \cos(2\pi\mathbf{B}\mathbf{x}))$$

- ▶ Passing network a subset of the Fourier basis functions. Same effect from:
 - ▶ Positional encoding
 - ▶ Fourier features
 - ▶ SIREN

Problem solved

Ground truth image



Standard fully-connected net



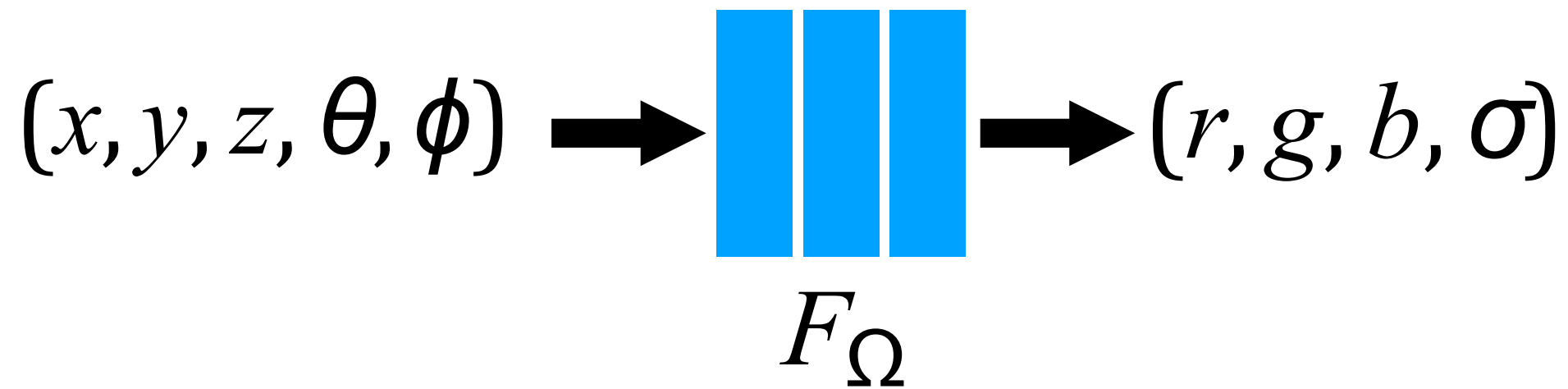
With "positional encoding"



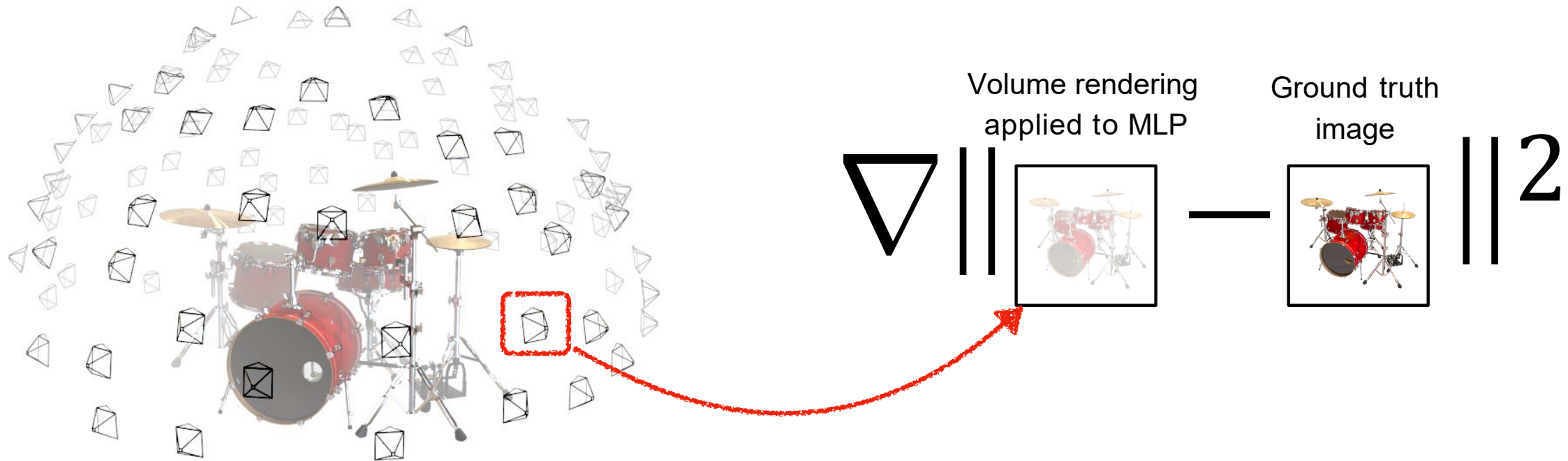
Overview

- ▶ Volumetric rendering math
- ▶ Neural networks as representations for spatial data
- ▶ **Neural Radiance Fields (NeRF)**

NeRF = volume rendering +
coordinate-based network



Train network to reproduce input views of scene using gradient descent



Visualizing view-dependent effects



Regular NeRF rendering



Manipulating input viewing directions

Visualizing learned density field as geometry



Regular NeRF rendering



Expected ray termination depth

Visualizing learned density field as geometry



Regular NeRF rendering



Expected ray termination depth

Any Questions?